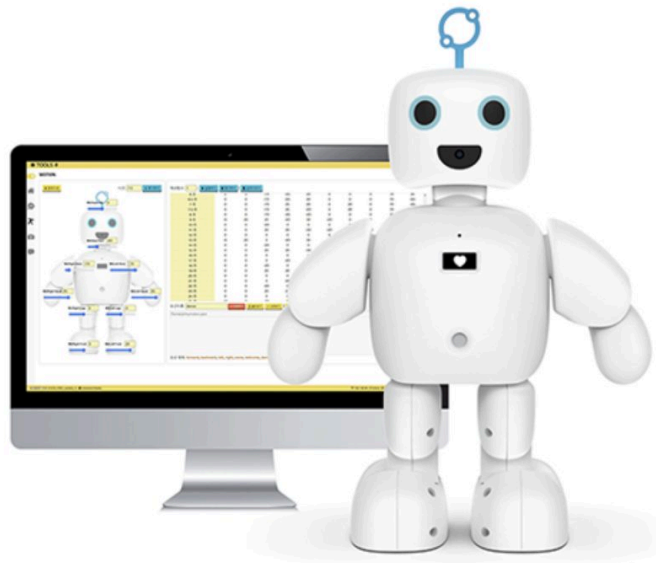


Pibo

Teacher's Guide

Block coding with the Pibo robot



Contents

1 Before the first lesson

2 Getting started with the IDE

3 Block reference

4 The ten built-in examples

5 Tools and Classifier

6 When something goes wrong

A Appendix A — Motors

B Appendix B — Motion names

C Appendix C — Voices

How to use this guide

Sections 1 and 2 are worth reading once before your first lesson. Section 3 is a reference — look things up rather than reading it through. Section 4 walks through the ten programs that are already on the robot, and is the fastest way to prepare a lesson.

1 Before the first lesson

Pibo is a small robot that children program by joining coloured blocks together on a screen. No typing is needed, and no internet connection is required — everything runs on the robot.

What you need

- A Pibo robot, charged or on its power adapter.
- A tablet or laptop with a web browser. Chrome is recommended.
- The classroom Wi-Fi router, switched on.

Turning Pibo on

1. Connect the power adapter, or press the power button if the robot is charged.
2. Pibo's eyes light up and it greets you. Starting up takes about a minute.
3. When the eyes settle and the chest screen shows an address, Pibo is ready.

Read the address from the chest screen

The chest screen shows three lines: **SN** (this robot's number), **I P** (the address to type into the browser) and **AP** (the Wi-Fi network it joined). Each robot has its own address, so read the screen of the robot in front of you rather than reusing a number from another lesson.

If the **I P** line is empty, the robot did not find the Wi-Fi network — see section 6.

```
SN: cd488e95
I P: 192.168.114.138
AP: pibo
```

Connected — open the **I P** address

```
SN: cd488e95
I P:
AP:
```

Not connected — see section 6

Opening the IDE

Type the address from the chest screen into the browser and press Enter. The block editor opens. Nothing needs to be installed.

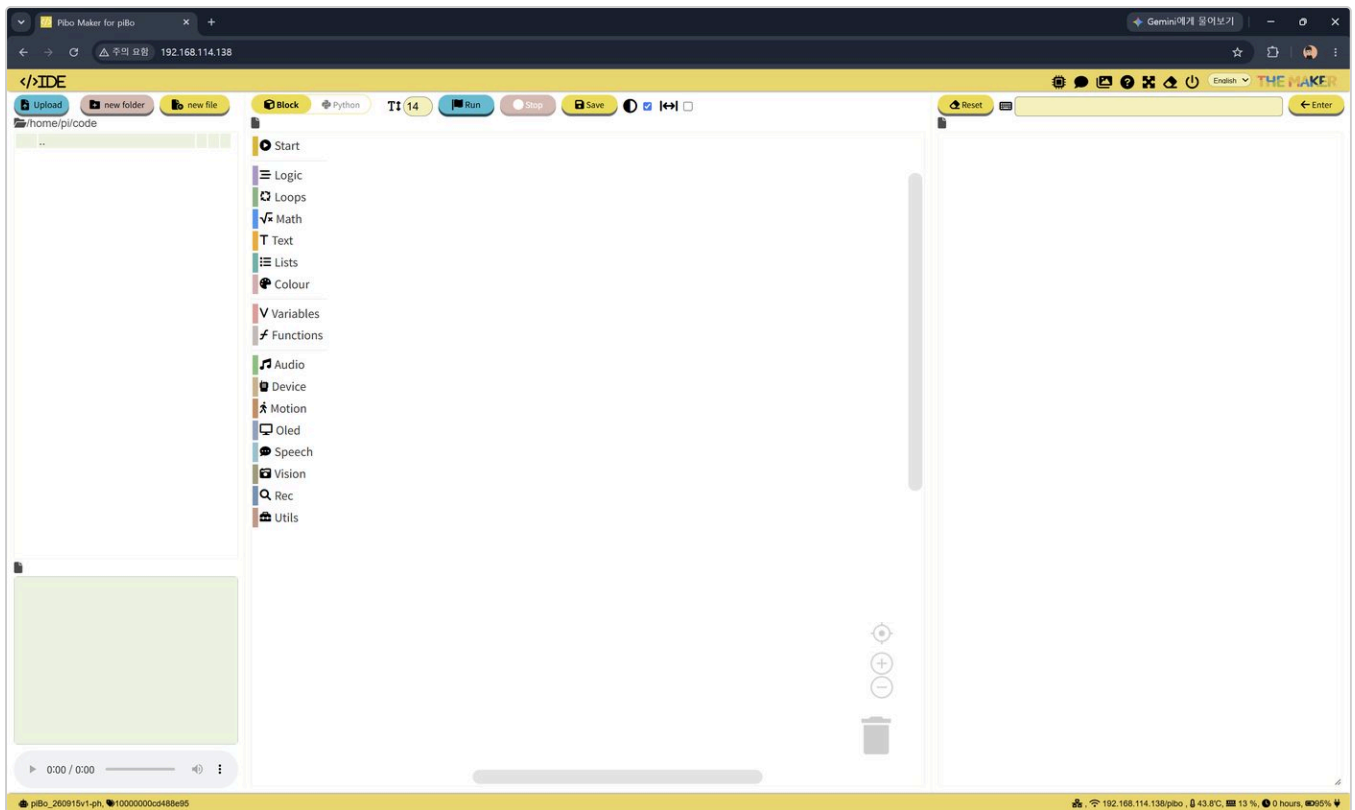
If the address does not work

Check that the tablet is connected to the same Wi-Fi network as the robot. If there is a network whose name begins with `pibo-`, see section 6 — the robot has not joined the classroom network and needs its Wi-Fi set again.

One robot, one tablet

Two tablets can open the same robot, but they share one workspace — what one child changes, the other sees. For group work this is useful; for individual work give each child their own robot.

2 Getting started with the IDE



IDE main screen, with the toolbox, canvas, terminal and toolbar marked

The screen

Toolbox	Down the left. Click a category to open its blocks, then drag one onto the canvas.
Canvas	The middle. Join blocks here. Drag a block away to detach it; drag it to the bin, or press Delete, to remove it.
Terminal	Along the bottom. Everything a Print block writes appears here, and so do error messages.
Image panel	Shows pictures sent by Display image variable on IDE . Used by all the camera examples.
Toolbar	Top right — run, stop, save, open, and the other tools.

Writing and running a program

1. Open the **Start** category and drag **When Run clicked** onto the canvas.
2. Add blocks underneath it. They click together like jigsaw pieces.
3. Press **Run**. Watch the Terminal for printed output.
4. Press **Stop** to end a program that is still running.

Blocks must be connected

Only blocks joined under **When Run clicked** will run. A stack left loose on the canvas is ignored — this is the most common reason a program “does nothing”.

Saving and opening

Use **Save** on the toolbar to keep a program on the robot, and **Open** to load it again or to load one of the ten examples. Files are stored on the robot, not on the tablet, so a child returning to the same robot finds their work waiting.

Changing the language

The language selector is in the IDE. Block names, menus and messages switch immediately; your blocks are not affected.

3 Block reference

Every block in the toolbox, in the order the categories appear. The wording matches the English interface exactly, so you can search for a block by reading it here.

A grey box  marks a place where something is filled in — a number, a piece of text, a colour, or another block plugged in.

Standard blocks and Pibo blocks

The first eight categories are the standard Blockly blocks used in most block-coding tools. The rest — Audio, Device, Motion, OLED, Speech, Vision, Rec and Utils — control the robot itself.

Start

Every program begins with this block. Blocks that are not connected below it will not run.

Block	What it does
When [Run] clicked	Run the code.

Logic

Standard Blockly blocks. Comparisons, AND/OR, NOT, true/false and the if / else-if / else structure. The **null** block stands for “no value”.

Loops

Standard Blockly blocks. Repeat a fixed number of times, repeat while a condition holds, count with a variable, walk through a list, and break / continue.

Math

Standard Blockly blocks. Numbers, the four operations, rounding, remainders, random numbers, trigonometry and list arithmetic.

Text

Standard Blockly blocks. Build, join, measure, search and edit text. **Print** writes a line to the Terminal at the bottom of the IDE.

Lists

Standard Blockly blocks. Create lists, read and write items by position, take sub-lists, split text into a list, sort and reverse.

Colour

Standard Blockly blocks. Pick a colour, make a random one, build one from red/green/blue values, or blend two colours. Colour blocks plug straight into the eye-LED blocks in **Device**.

Variables

Blockly creates these blocks for you. Click **Create variable...**, give it a name, and the *set* and *get* blocks appear in this category.

Functions

Blockly creates these blocks for you. Define a named block of code once, then call it from anywhere. Use it when the same steps repeat in several places.

Audio

Play sound files and record from the microphone. Files live in the robot's audio folder and appear in the file browser on the left of the IDE.

Block	What it does
Play audio with volume	Select and play audio at set volume.
Play . audio with volume	Enter music file name to play.
Stop audio	Stop the audio currently playing.
Record for seconds and save to .	Record for a set time and save as a file.

Device

The eye LEDs and the sensors built into Pibo. The *Check...* blocks report a value, so plug them into a comparison or a **Print** block.

Block	What it does
Turn on eye LEDs	Turn on eye LED
Turn on eye LEDs with color variable	Turn on eye LED with a color variable.
Check adapter	Check adapter connection status.
Check battery	Check battery status.
Check system message	Check system messages.
Check for human presence	Detect presence of people.
Check touch	Detect touch.
Check button	Check button status.

Motion

Move the servo motors. Use a ready-made motion by name, run a motion you recorded yourself in **Tools**, or drive one motor at a time.

Block	What it does
Get motion list	Retrieve list of available motions.
Get my motion list	Retrieve list of custom motions.
Execute motion times	Execute a chosen motion a specified number of times.

Block	What it does
Execute motion times	Execute an entered motion a specified number of times.
Execute my motion times	Execute a custom motion.
Initialize motion	Initialize motion settings.
Move motor by degrees	Move a specified motor to a certain degrees.
Set motor speed to	Set the speed of the motor.
Set motor acceleration to	Set the acceleration of the motor.
Move total motors degrees time	Moves all motors to a certain degrees by the specified time.

Motion names are listed in Appendix B; motor numbers and their limits in Appendix A.

OLED

Draw on the small screen on Pibo's chest. Drawing blocks only prepare the picture — nothing appears until you use **Show OLED**.

Block	What it does
Set OLED font size to	Set the font size on the OLED display.
Save to OLED at	Display text on the OLED screen.
Save OLED image	Display dynamically selected image on OLED.
Save OLED . image	Display a specified image on OLED.
Save OLED image data	Display a specified image data on OLED.
Save rectangle to OLED at ,	Draw a rectangle on OLED.
Save ellipse to OLED at ,	Draw an ellipse on OLED.
Save line to OLED at ,	Draw a line on OLED.
Invert OLED	Invert colors on the OLED screen.
Show OLED	Display content on OLED screen.
Clear OLED	Clear the OLED screen.

Speech

Make Pibo talk, and use the on-board language model. Two voices are available: the on-device voice (natural, handles English and Korean automatically) and the Espeak voice (robotic, very fast).

Block	What it does
Save with voice to .	Text to Speech.(ondevice)
Say with voice at volume	Text to Speech and Say that.(ondevice)
Save with Espeak voice to .	Text to Speech.(Espeak)
Say with Espeak voice at volume	Text to Speech and Say that(espeak)

Block	What it does
Start LLM server	Start LLM server.
Respond to , System prompt	Call LLM Server. (prompt/system prompt)
Stop LLM server	Stop LLM server

Voice codes are listed in Appendix C.

Vision

Take pictures with the camera and edit them. Most blocks take an *image* and give back a changed image, so you normally store the result in a variable.

Block	What it does
Take a picture	Capture an image.
Load image file	Load a specified image file.
Load image file .	Load an image file.
Create Color matte	Create Color Matte
Save image to .	Save as an image file.
Display image variable on IDE	Display an image variable in IDE.
Display image variable on OLED	Display an image variable in OLED.
Display a square with color and thickness at coordinates of image .	Displays a square in the image.
Display a circle with radius , color , thickness at coordinates of image .	Displays a circle in the image.
Display a line with color and thickness at coordinates from to of image .	Displays a line in the image.
Display with size and color at coordinates and of image .	Display text on the image.
Convert image to	Transform image style.
image resize to width height	Resize the image.

Rec (Recognition)

Recognition — the AI blocks. Faces, objects, QR codes, hand gestures, body pose, markers, and your own image classifier trained in **Classifier**. All of it runs on the robot itself; no internet is needed.

Block	What it does
Find faces in image	Detect and identify faces in the specified image.
Show face in the image	Show faces in the image.

Block	What it does
Analyze face ☐ of image ☐ (age, gender, emotion)	Analyze face in image(age, gender, emotion)
Show face analysis ☐ in the image ☐	Show face analysis in the image
Find face ☐ landmark in the image ☐	Find face landmark in the image
Show face landmark ☐ in the image ☐	Show face landmark in the image.
Face dictionary	Face Trained Data Dictionary
Train image ☐ face ☐ to name ☐	Train face from image.
Delete face name ☐ from face training data	Delete a specific face from training faces.
Recognize who is image ☐ face ☐	Recognize who faces are in an image.
Save face training data to file ☐ ☐	Save face training data to file.
Load face training data from file ☐ ☐	Load face training data from file.
Analyze face direction/distance in the image ☐	Analyze face direction/distance in the image.
Show face direction/distance ☐ in the image ☐	Show face direction/distance in the image
Load Object detection model ☐ ☐ .onnx	Load Object detection model
Find objects in the image ☐	Find objects in the image.
Find objects in the image ☐ (Raw)	Find objects in the image (RAW)
Show objects ☐ in the image ☐	Show objects in the image.
Find QR in the image ☐	Find QR codes in the image.
Find QR in the image ☐ (RAW)	Find QR codes in the image. (RAW)
Show QR ☐ in the image ☐	Show QR codes in the image.
Detect poses in image ☐	Identify and analyze human poses in the specified image.
Show pose data ☐ in the image ☐	Show pose data in the image.
Analyze pose results in ☐ and save to ☐	Evaluate and interpret pose data from the image and save the results.
Setting tracker at coordinates ☐ ☐ , ☐ ☐ of image ☐	Set a tracker for a specific location in the image.
Track thing in the image ☐	Track thing in the image.
Show tracking result ☐ in the image ☐	Show tracking result in the image.
Load Hand gesture model ☐	Load Hand gesture model.
Load Hand gesture model ☐ ☐ .task	Load Hand gesture model
Recognize Hand gesture in the image ☐	Recognize Hand gesture in the image.
Show hand gesture ☐ in the image ☐	Show hand gesture in the image.
Detect Marker length ☐ in image ☐	Detect Marker in the specified image.

Block	What it does
Show marker in the image	Show marker in the image.
Set My Image model for	Configure My Image model for image classification.
Classify image using My Image model	Use My Image model to classify the image.

Utils

Small helpers that are not part of standard Blockly: waiting, the clock, dictionaries, type conversion and angle maths.

Block	What it does
Delay for seconds	Pause the process for a specified duration of seconds.
Get time value	Retrieve the current time value.
Get current time	Obtain the current time.
Check if is included in	Verify if an element is included in a list or array.
Get value of key from dictionary	Retrieve the value associated with a specific key in a dictionary.
Append { : } to dictionary	Add a new key-value pair to an existing dictionary.
Create empty dictionary	Generate a new empty dictionary.
set array [: , :] to	Set the values of the specified range in the array.
Check if exists	Confirm the existence of a file or directory.
Convert to String type	Change a variable's type to String.
Convert to type	Convert a variable to a specific numeric type. (Integer or Float)
Get angle of - -	Get angle

4 The ten built-in examples

These ten programs are already on every robot. Open them with **Open** on the toolbar. They are the quickest way to prepare a lesson: run one, read it together, then change one number and run it again.

Example	Teaches
Basic	printing, if / else
Device — sensors and eye LEDs	sensors, loops, LEDs
Times table	nested loops, variables
Motion	calling robot motions
Speech — text to speech	text to speech, two voices
Face — direction and distance	camera loop, dictionaries
Face — age, gender, emotion	face analysis, lists
Hand gestures	a loaded AI model
Markers	measurement from a camera
Objects and QR codes	two recognisers at once

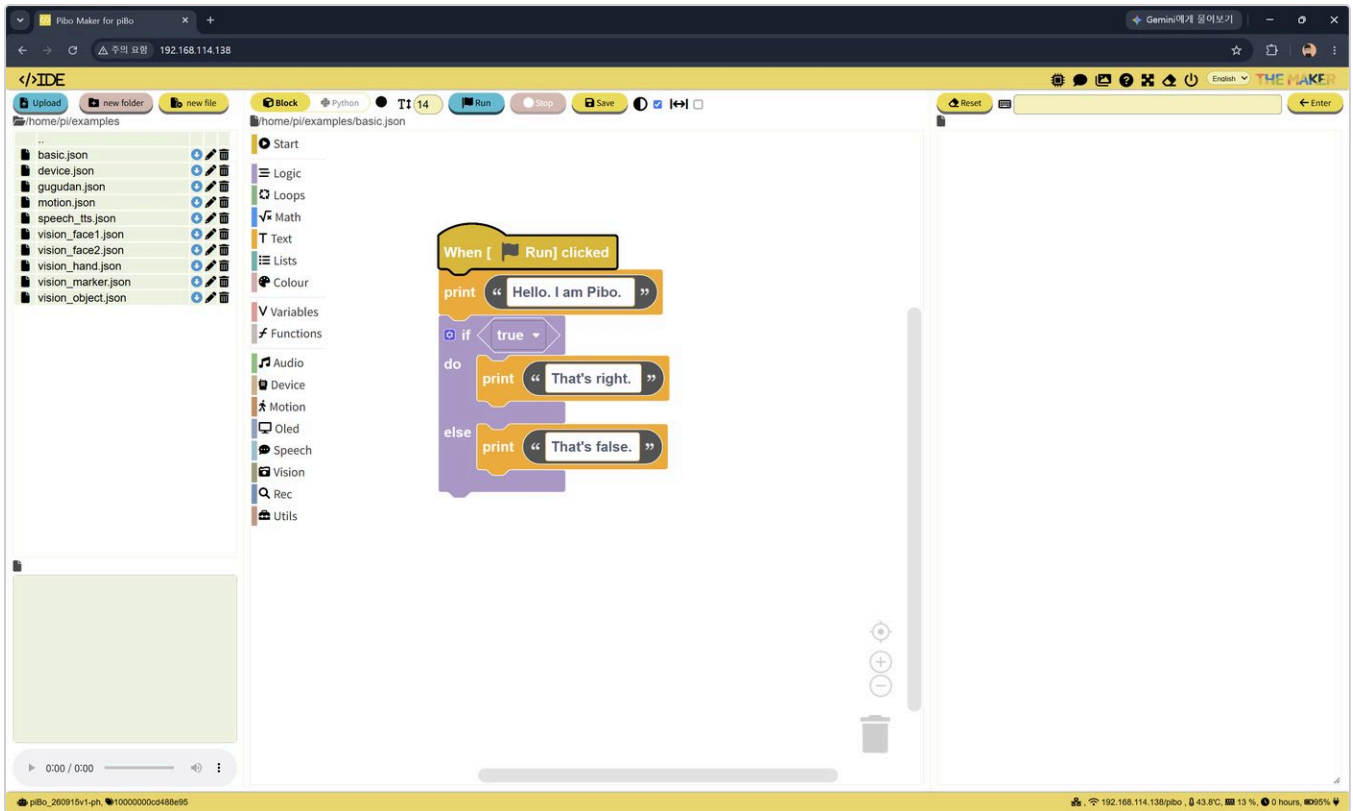
A lesson shape that works

Run the example first without explaining it, and let the class see what it does. Then open the blocks and read them together. Then change exactly one thing — a number, a word, a colour — and run it again. Children remember the change far better than the explanation.

Basic

basic.json

Your first program. Prints a line of text, then shows how an **if / else** block chooses between two messages.



IDE canvas with basic.json loaded

What the blocks do

1. Print *Hello. I am Pibo.* to the Terminal.
2. Ask whether **true** is true — it always is.
3. Because the answer is yes, print *That's right.* The *else* branch never runs.

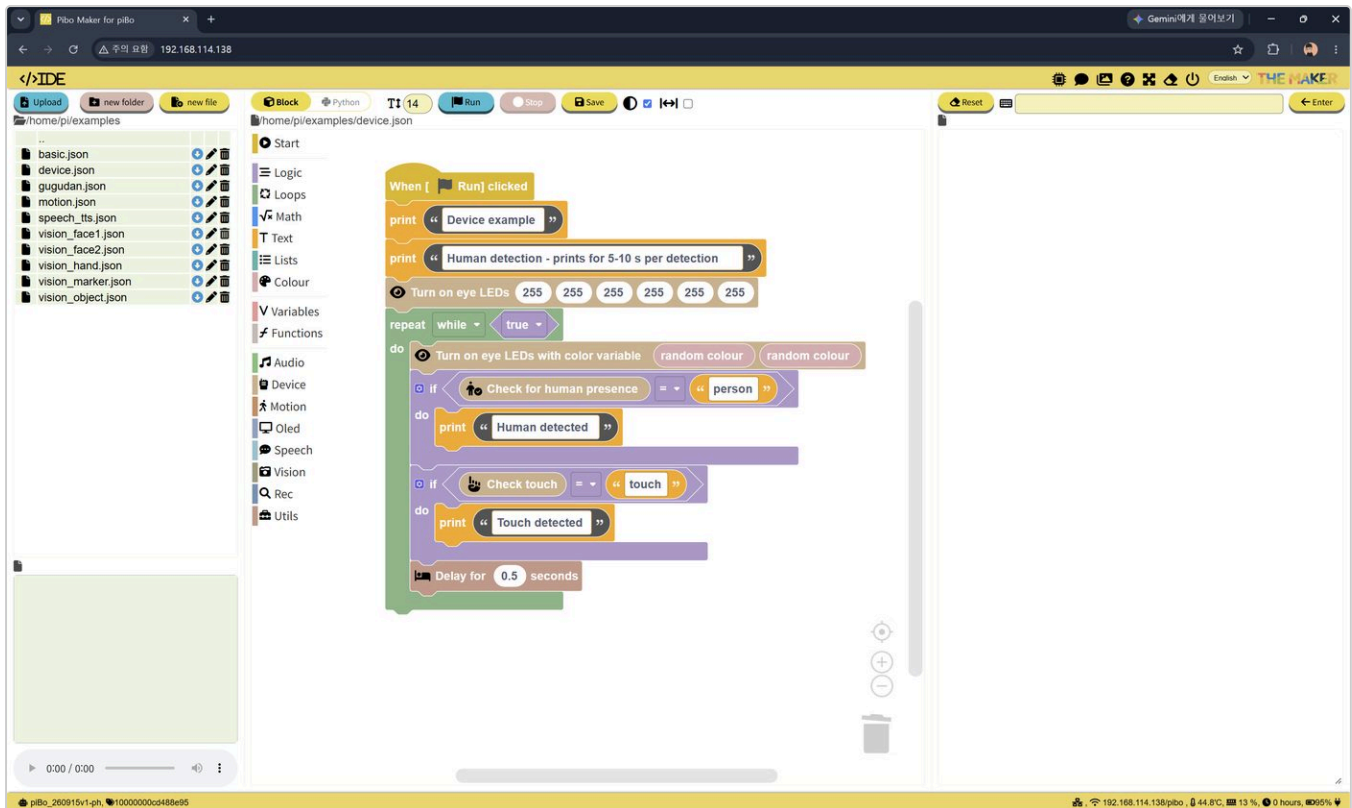
In the classroom

Change **true** to **false** and run it again. Now the other message appears. This is the smallest possible demonstration of a decision in code.

Device — sensors and eye LEDs

device.json

Turns the eye LEDs on, then loops forever reading the human-presence sensor and the touch sensor.



IDE canvas with device.json loaded

What the blocks do

1. Print a title line.
2. Turn both eye LEDs full white (255, 255, 255 on each side).
3. Start a loop that repeats while **true** — that is, forever.
4. Set both eyes to a random colour.
5. If the presence sensor returns *person*, print *Human detected*.
6. If the touch sensor returns *touch*, print *Touch detected*.
7. Wait 0.5 seconds, then repeat.

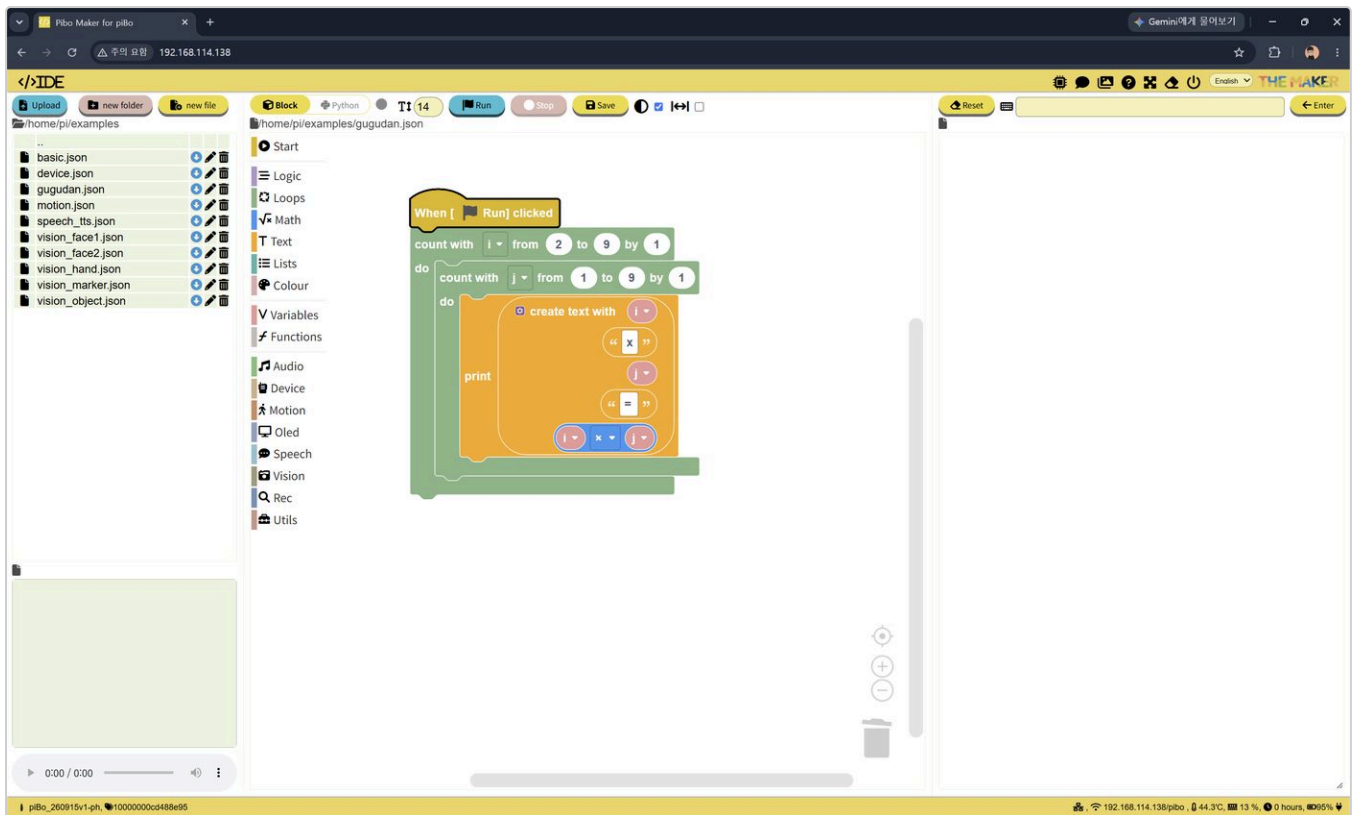
In the classroom

The loop never ends by itself. Press **Stop** in the IDE to end it. Note how each sensor block is compared against a word using the = block from **Logic**.

Times table

gugudan.json

Two nested loops print a multiplication table. No robot hardware is used, so it works as a pure programming lesson.



IDE canvas with gugudan.json loaded

What the blocks do

1. The outer loop counts i from 1 to 9.
2. For each i , the inner loop counts j from 1 to 9.
3. Print $i \times j =$ the product, joined into one line of text.

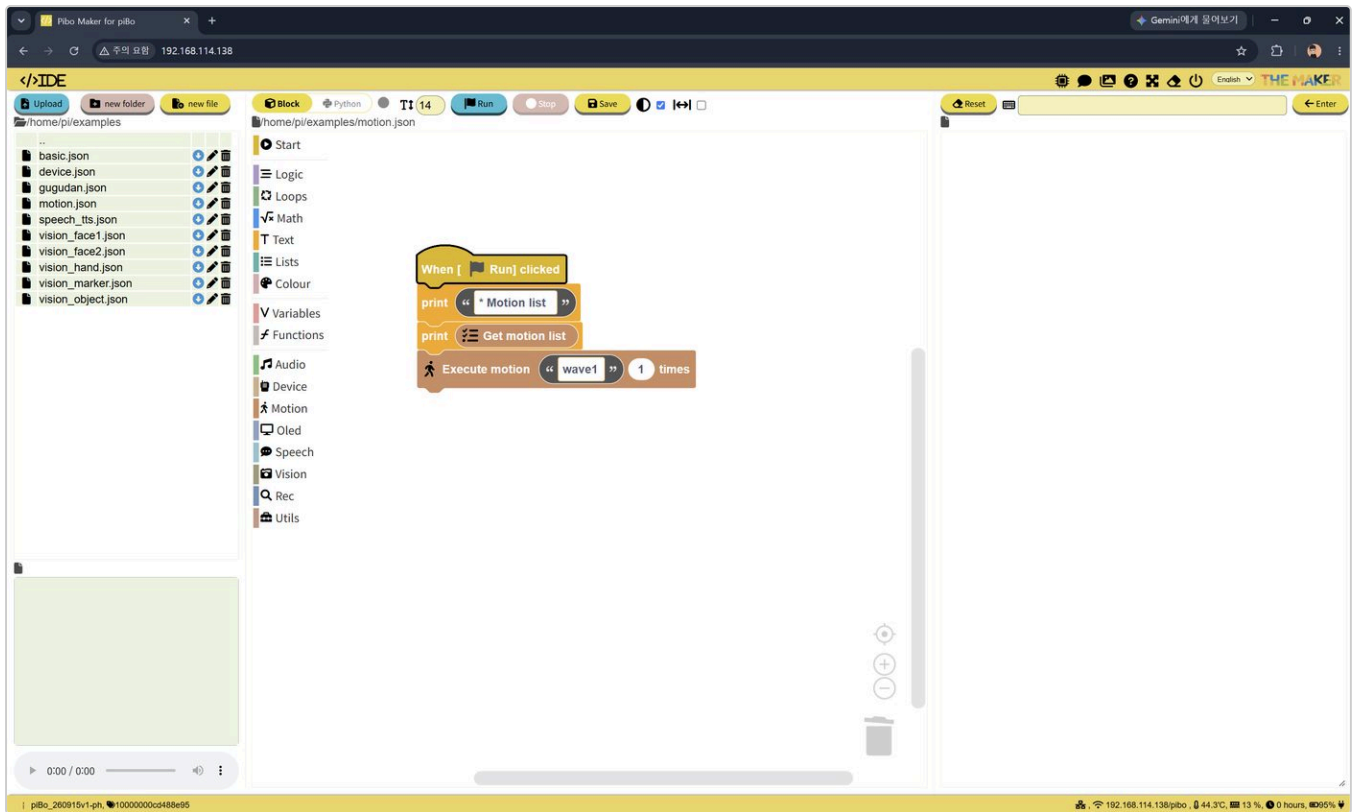
In the classroom

A good first look at nested loops. Ask the class to predict how many lines will be printed before running it.

Motion

motion.json

Lists the motions the robot knows, then performs one.



IDE canvas with motion.json loaded

What the blocks do

1. Print a title line.
2. Print the result of **Get motion list** — every motion name the robot can perform.
3. Run the motion *wave1* once.

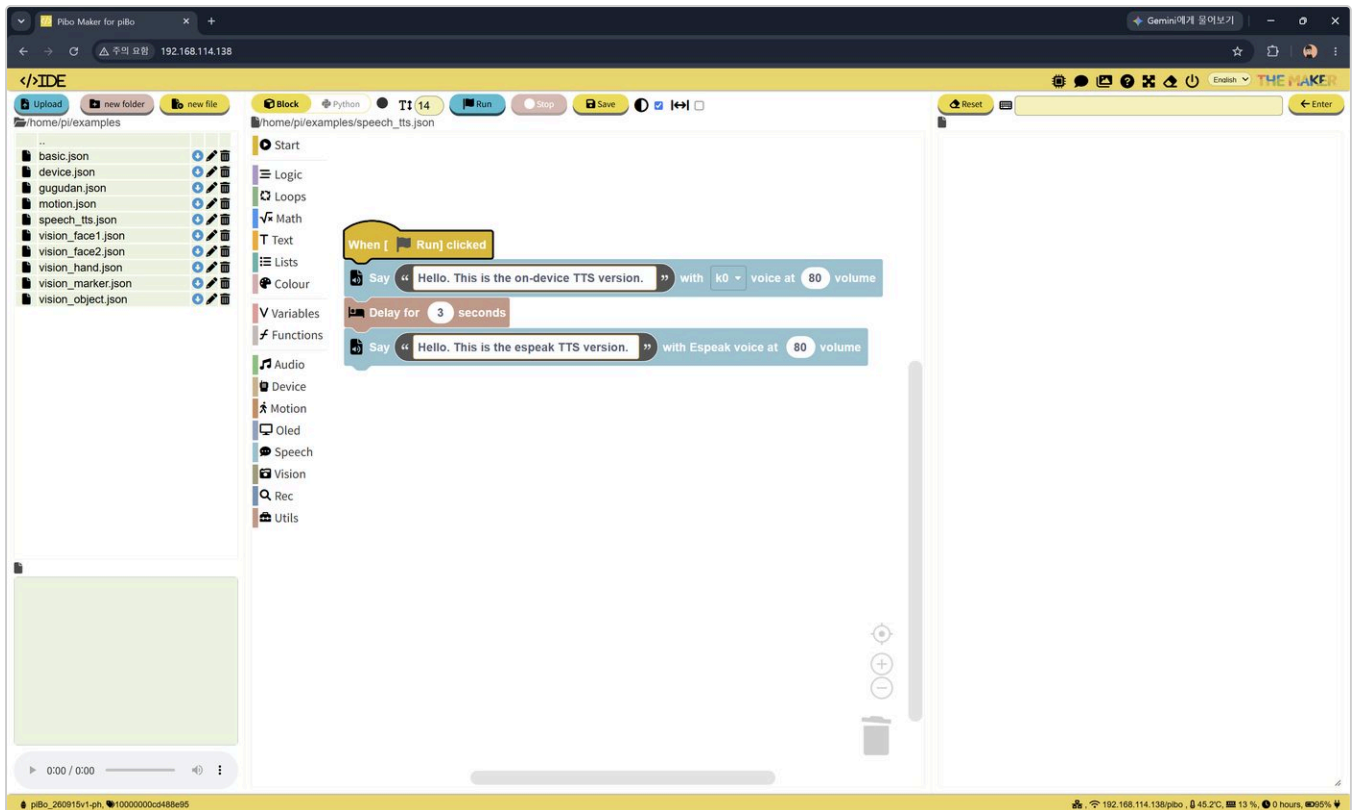
In the classroom

Replace *wave1* with any name from the printed list. The full list is also in Appendix B. Make sure Pibo has room to move before running motion blocks.

Speech — text to speech

speech_tts.json

Says the same sentence with each of the two voices so the class can hear the difference.



IDE canvas with speech_tts.json loaded

What the blocks do

1. Say a sentence with the on-device voice *m1* at volume 80.
2. Wait 3 seconds.
3. Say a sentence with the Espeak voice at volume 80.

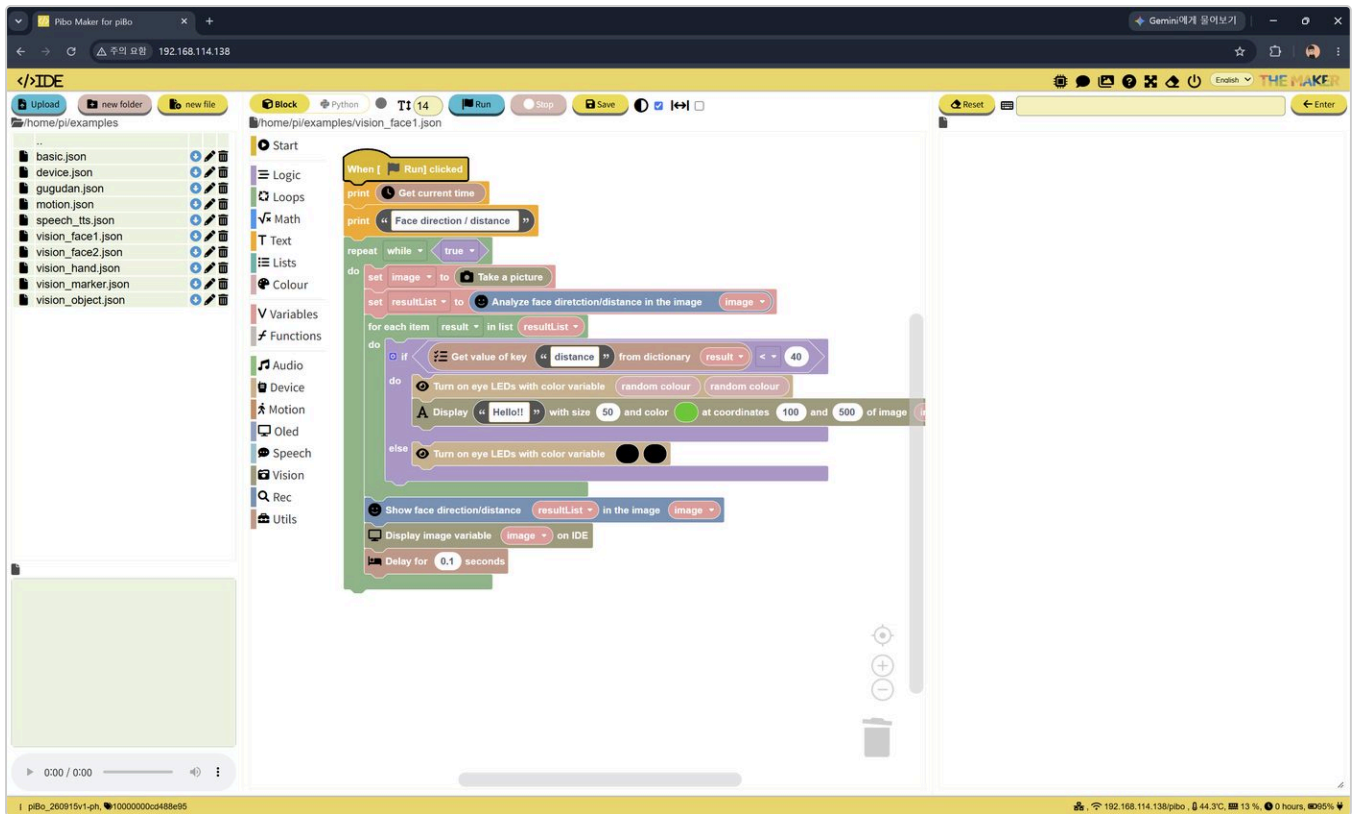
In the classroom

The on-device voice sounds natural and detects the language automatically. Espeak is robotic but starts instantly. Voice codes are listed in Appendix C.

Face — direction and distance

vision_facel.json

Watches through the camera and reacts when someone comes close.



IDE canvas with vision_facel.json loaded

What the blocks do

1. Take a picture and store it in *image*.
2. Analyse face direction and distance; store the list in *resultList*.
3. For each face found, read the *distance* value from the result.
4. If the distance is under 40, flash the eyes a random colour and write *Hello!!* onto the picture.
5. Draw the analysis onto the picture and show it in the IDE.
6. Wait 0.1 s and repeat.

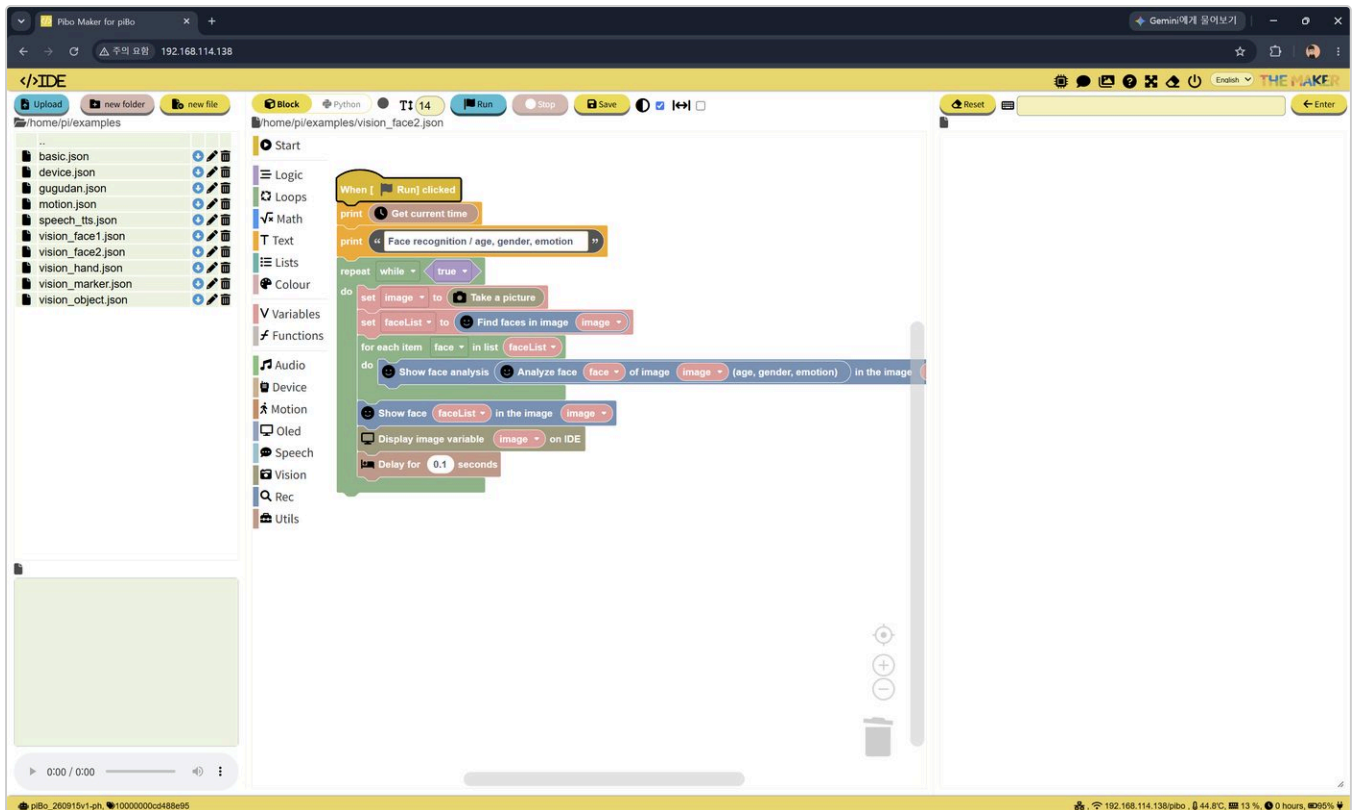
In the classroom

The live picture appears in the image panel of the IDE. Try changing 40 to a larger number so Pibo greets people from further away.

Face — age, gender, emotion

vision_face2.json

Finds every face in the camera view and labels it.



IDE canvas with vision_face2.json loaded

What the blocks do

1. Take a picture and store it in *image*.
2. Find all faces; store them in *faceList*.
3. For each face, analyse age, gender and emotion, and draw the result on the picture.
4. Draw boxes around all the faces.
5. Show the picture in the IDE, wait 0.1 s and repeat.

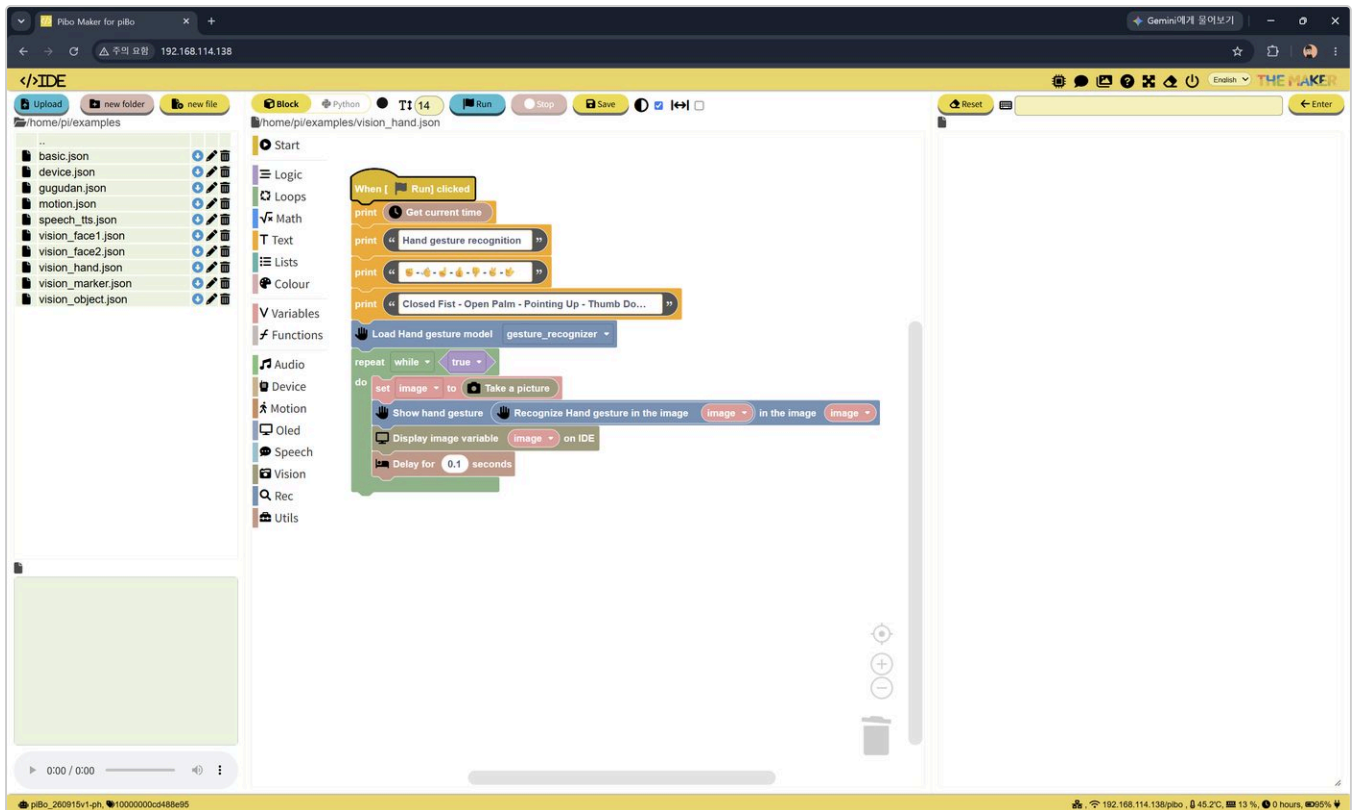
In the classroom

Estimates are approximate and will sometimes be wrong — that is worth discussing with the class rather than hiding.

Hand gestures

vision_hand.json

Recognises seven hand signs from the camera.



IDE canvas with vision_hand.json loaded

What the blocks do

1. Print the seven gestures the model knows.
2. Load the hand-gesture model.
3. Take a picture, recognise the gesture, draw the result on the picture.
4. Show the picture in the IDE, wait 0.1 s and repeat.

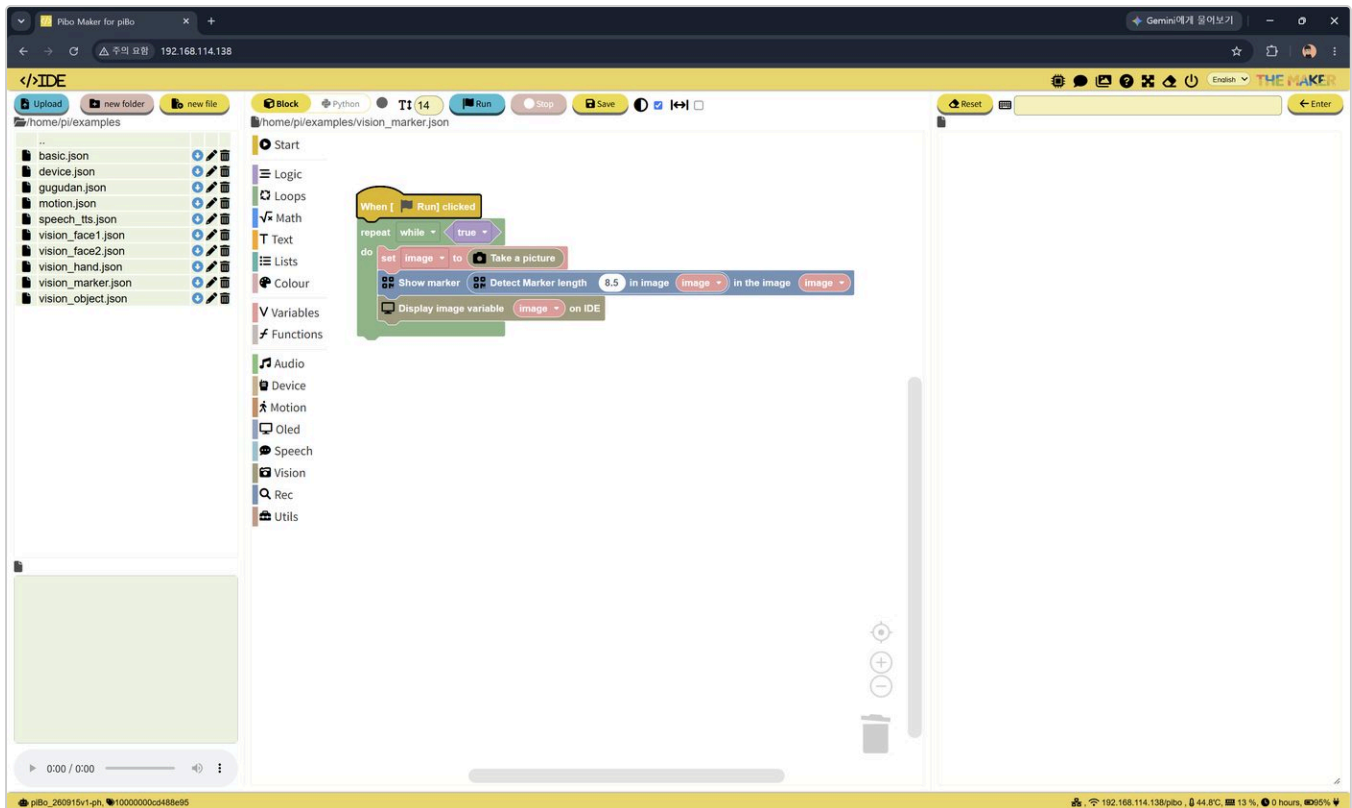
In the classroom

The seven signs are Closed Fist, Open Palm, Pointing Up, Thumb Down, Thumb Up, Victory and I Love You. Hold your hand still and well lit.

Markers

vision_marker.json

Detects printed ArUco markers and measures them.



IDE canvas with vision_marker.json loaded

What the blocks do

1. Take a picture.
2. Detect markers, told that each printed marker is 8.5 cm across.
3. Draw the markers on the picture and show it in the IDE.

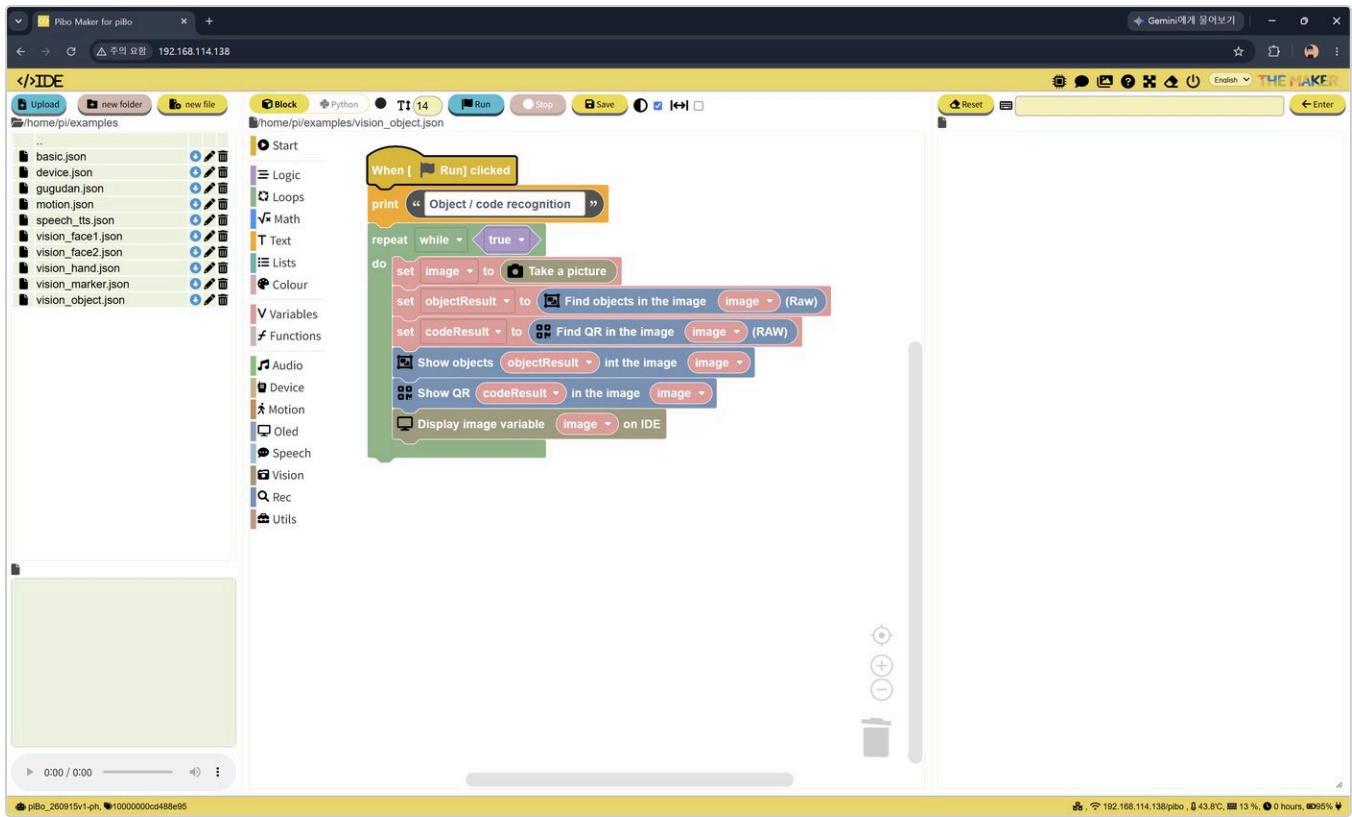
In the classroom

The 8.5 number must match the real printed size, otherwise the distance readings are wrong. Print the markers at a known size and measure one with a ruler.

Objects and QR codes

vision_object.json

Recognises everyday objects and reads QR codes at the same time.



IDE canvas with vision_object.json loaded

What the blocks do

1. Take a picture.
2. Find objects in it; store the raw result.
3. Find QR codes in it; store the raw result.
4. Draw both results on the picture and show it in the IDE.

In the classroom

Print a few QR codes beforehand. Objects are recognised from a fixed list — if something is not recognised, that is a limit of the model, not a fault.

5 Tools and Classifier

Two extra programs open from the IDE toolbar. Each one takes over the robot while it is open, so close the tab when you have finished.

Tools

Used to record things the block editor then plays back.

- **Motion** — pose the robot by hand or with sliders, capture each pose, and save the sequence as your own motion. It then appears in the **Execute my motion** block.
- **Wait until it is ready.** When Tools opens, the motor button reads **OFF** for about twenty seconds while the robot prepares itself. Begin once it turns to **ON**.
- **Recording** — record a sound through the microphone and save it as a file, which the **Play audio** block can then use.

Tools — motion editor

Time	M0	M1	M2	M3	M4	M5	M6	M7	M8	M9
0 s	0	0	-70	-25	0	0	0	0	70	25
0.3 s	10	0	-70	-25	0	0	20	0	70	25
0.6 s	10	0	-80	-25	20	0	20	-30	60	25
0.9 s	10	-30	-80	-25	20	0	20	-30	60	25
1.2 s	10	-30	-80	-25	20	0	0	-30	60	25
1.5 s	-20	-30	-80	-25	20	0	-10	-30	60	25
1.8 s	-20	30	-60	-25	-20	0	-10	-30	80	25
2.1 s	-20	30	-60	-25	-20	0	-10	30	80	25
2.4 s	0	30	-60	-25	-20	0	0	30	80	25

Motion name: Register Load Remove Remove all Export Import

Example: left, left_half, right, right_half, forward1, backward1, step1, cheer1, cheer2, wave1, think1, wake_up3, yes_h, no_h, head_h, spin_h, clapping1, handshaking, greeting, hand1, foot1, speak1, speak2, welcome, dance1, dance2, dance3, dance4, dance5

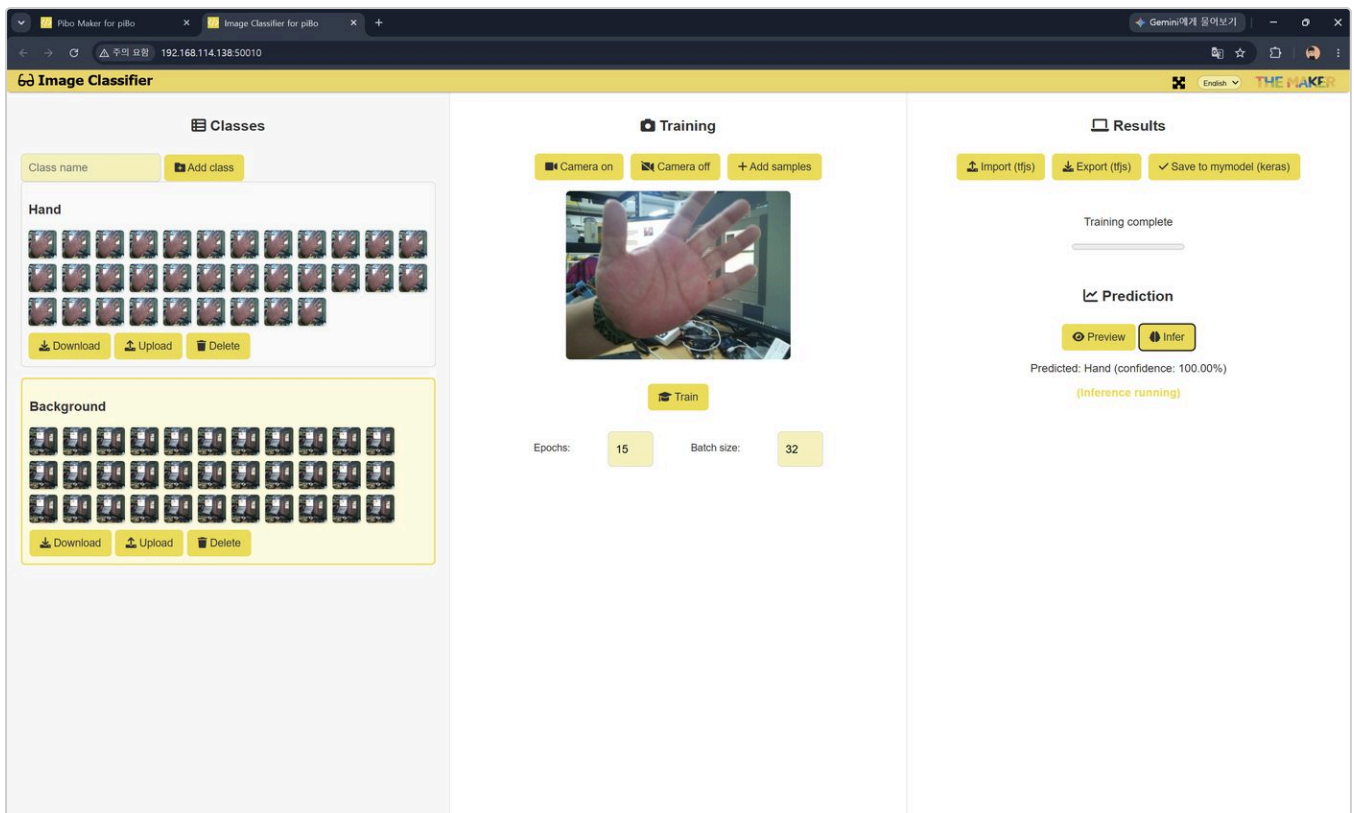
Tools — motion editor

Classifier

Trains the robot to tell apart categories of pictures that you choose — for example three kinds of fruit, or tidy versus untidy desks.

1. Create a category and give it a name.
2. Hold an object in front of the camera and capture a number of images. Twenty or more per category works well.
3. Repeat for each category.
4. Train, then test by holding up an object.

Once trained, the model is used from the block editor with **Set My Image model** followed by **Classify image using My Image model**, in the **Rec** category.



Classifier — training screen

Close the tab when you finish

Tools and Classifier each hold the camera and the motors. If a camera block seems frozen, or motion blocks do nothing, a Tools or Classifier tab is probably still open somewhere.

6 When something goes wrong

Symptom	What to do
The chest screen shows no IP address	Pibo could not find the classroom Wi-Fi, so it has started a network of its own named <code>pibo-</code> followed by the SN shown on the chest screen. This is an administrator job rather than a classroom one — report the robot instead of trying to fix it during a lesson.
The IDE page will not open	Check that the tablet is on the same network as the robot. Then reload with Ctrl + Shift + R — an ordinary reload can serve an old cached page.
The tablet keeps dropping the robots	The robot network normally has no internet, and tablets and laptops try to leave a network that has none. The tablet has moved to another Wi-Fi network by itself. Reconnect it to <code>pibo</code> , and ask your administrator to turn off automatic joining for the other networks — it is a one-off setting per device.
Windows says “No internet, secured”	That is normal and nothing is wrong. Everything in the lesson runs on the robot itself, so no internet is needed. Do not switch to another network to make the warning go away.
Code runs but Pibo does nothing	Blocks must be attached under When Run clicked . A stack sitting loose on the canvas is ignored.
A motion block does nothing	Check the spelling of the motion name against Get motion list , and make sure nothing is blocking the robot's arms or legs.
The camera image is frozen	Only one tool may use the camera at a time. Close the Classifier and Tools tabs, then run your program again.
Sound is too quiet or too loud	Every speech and audio block has a volume input. 80 is a good classroom level.
A program will not stop	Loops that repeat <i>while true</i> run forever by design. Press Stop in the IDE.
I want to start over	Factory reset puts the example files back and deletes saved work, recordings and trained models. It also resets the Wi-Fi setting and switches the robot off , so unless your network is named <code>pibo</code> somebody will have to set the Wi-Fi again before the robot can be used. Ask whoever set the robots up before using it.

Before calling for help

Note the serial number shown at the bottom of the IDE, what you were doing, and anything printed in red in the Terminal. That is usually enough to identify the problem without visiting the classroom.

Appendix A Motors

Motor numbers used by **Move motor by degrees**, and how far each one turns. Left and right are Pibo's own left and right, not yours.

No.	Position	Range
0	Right foot	$\pm 25^\circ$
1	Right leg	$\pm 35^\circ$
2	Right arm	$\pm 80^\circ$
3	Right hand	$\pm 30^\circ$
4	Neck (pan)	$\pm 50^\circ$
5	Head (tilt)	$\pm 25^\circ$
6	Left foot	$\pm 25^\circ$
7	Left leg	$\pm 35^\circ$
8	Left arm	$\pm 80^\circ$
9	Left hand	$\pm 30^\circ$

Stay inside the range

Asking a motor to go past its limit strains it and can make the robot fall over. Keep a clear space around Pibo whenever motion blocks are running.

Appendix B Motion names

Names accepted by **Execute motion**. The robot can also list them at run time with **Get motion list**.

backward1	foot1	left_half	step1
backward2	foot2	look_l	step2
boring1	foot3	look_r	stop
boring2	forward1	lookup	stop_body
bow	forward2	no_h	think1
breath1	greeting	right	think2
breath2	hand1	right_half	think3
breath3	hand2	sad1	think4
breath_long	hand3	sad2	wake_up1
cheer1	hand4	sad3	wake_up2
cheer2	handshaking	sleep	wake_up3
cheer3	handup_l	speak1	wave1
clapping1	handup_r	speak2	wave2
clapping2	happy1	speak_l1	wave3
dance1	happy2	speak_l2	wave4
dance2	happy3	speak_n1	wave5
dance3	head_h	speak_n2	wave6
dance4	hey1	speak_q	welcome
dance5	hey2	speak_r1	yes_h
excite1	hifive	speak_r2	
excite2	left	spin_h	

Motions you record yourself

Motions made in **Tools** are not in this list. Use **Execute my motion** for those, or **Get my motion list** to see their names.

Appendix C Voices

Codes for **Say ... with ... voice**, the on-device voice. The voice detects the language of the text by itself.

Code	Voice
m1	Male 1
m2	Male 2
m3	Male 3
m4	Male 4
m5	Male 5
f1	Female 1

Code	Voice
f2	Female 2
f3	Female 3
f4	Female 4
f5	Female 5

The Espeak blocks have a single fixed voice and take no voice code. Espeak sounds robotic but begins speaking instantly.